

VCO2IL Computer Vision Term Report

Simone B. Feldbacher, Andreas Schrattenecker

July 10, 2026

Abstract

This report presents two classical computer vision exercises and a short literature review on hand gesture and sign language recognition. The exercise part covers panorama stitching, feature matching, camera calibration, pose estimation, DCT-based image compression, and implicit neural representations. The research part summarizes the current state of hand gesture recognition and sign language translation, with attention to vision-based, sensor-based, and multimodal approaches, as well as current challenges in robustness, data availability, and real-time performance.

Contents

I	Class Exercises	4
1	Exercise 1	5
1.1	Introduction	5
1.2	Panorama Stitching	5
1.3	Feature Matching	6
1.4	Camera Calibration	8
1.5	Extrinsic Estimation	9
1.6	Conclusion	10
2	Exercise 2	11
2.1	Introduction	11
2.2	Frequency Domain Compression	11
2.2.1	Implementation	11
2.2.2	Results and Analysis	12
2.3	Implicit Neural Representations	14
2.3.1	Concept	14
2.3.2	Comparison	14
2.4	Conclusion	14
II	Related Work Research - Real time Hand Gesture Recognition	15
3	Introduction	16
4	Hand Tracking	17
4.1	Overview	17
4.2	Vision-Based Recognition	17
4.2.1	Processing Pipeline	17
4.2.2	From Hand-Crafted Features to Deep Learning	18
4.3	Sensor-Based Recognition	18
4.4	Multimodal Fusion	18
4.5	Open Challenges	18

Contents	3
5 Vision Based Sign Language Recognition	20
5.1 Overview	20
5.2 The Challenge of Sign Language Recognition	20
5.2.1 Datasets	20
5.3 Current Approaches	21
5.3.1 Sign Language Transformers	21
5.3.2 Large Sign Language Models	22
6 Conclusion and Outlook	23
References	24

Part I

Class Exercises

Chapter 1

Exercise 1

1.1 Introduction

This chapter summarizes the implementation and evaluation of several computer vision algorithms using the drone image datasets provided. The implemented methods include panorama stitching, feature matching, camera calibration, and extrinsic pose estimation.

1.2 Panorama Stitching

Panorama stitching was performed using OpenCV's built-in `Stitcher` class. The implementation is shown in Program 1.1 and Program 1.2.

Program 1.1: Panorama stitching using OpenCV

```
stitcher = cv2.Stitcher_create()

_, panorama = stitcher.stitch(images["RGB"])

imshow(panorama)
```



Figure 1.1: Panorama generated from the RGB images.

Program 1.2: Thermal panorama stitching

```
_, panorama = stitcher.stitch(images["Thermal"])  
  
imshow(panorama)
```

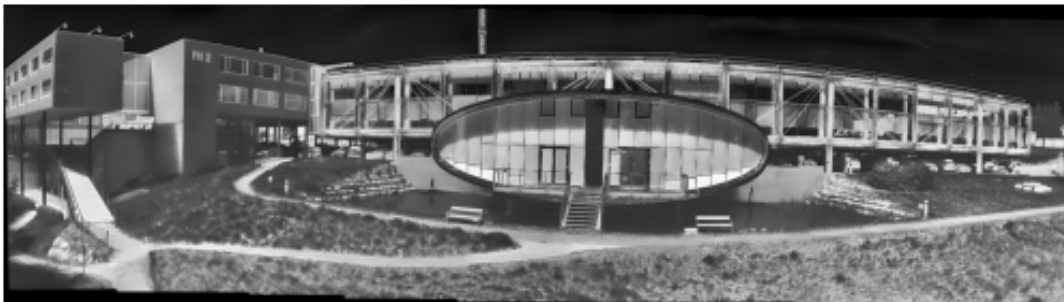


Figure 1.2: Panorama generated from the thermal images.

The panorama was successfully generated for both the RGB and thermal image sets. The resulting panoramas show that the provided image sequence contains sufficient overlap for automatic image alignment.

1.3 Feature Matching

To compare feature descriptors for thermal and RGB images, SIFT, ORB, and AKAZE were evaluated on all four provided datasets. The features were detected independently in both images, matched using a brute-force matcher, and filtered using the Lowe ratio test. The number of detected keypoints and the remaining good matches were recorded for each descriptor.

Program 1.3: Feature detection and matching

```

thermal = cv2.cvtColor(images["Thermal"][0], cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(images["RGB"][0], cv2.COLOR_BGR2GRAY)

kp_t, des_t = detector.detectAndCompute(thermal, None)
kp_r, des_r = detector.detectAndCompute(rgb, None)

bf = cv2.BFMatcher(norm_type)
matches = bf.knnMatch(des_t, des_r, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

```

The same procedure was applied to all four scenes using the SIFT, ORB, and AKAZE descriptors. The resulting number of key points detected and good matches is summarized in Table 1.1.

	descriptor	scene	keypoints_thermal	keypoints_rgb	good_matches
0	SIFT	ellipse	18860	60270	74
1	SIFT	FH3	2656	19126	40
2	SIFT	forest	3368	26255	15
3	SIFT	hut	11806	120369	36
4	ORB	ellipse	20000	20000	88
5	ORB	FH3	8987	20000	45
6	ORB	forest	7694	19441	60
7	ORB	hut	19189	20000	41
8	AKAZE	ellipse	10521	21427	43
9	AKAZE	FH3	1455	10028	8
10	AKAZE	forest	394	2926	3
11	AKAZE	hut	5220	35822	34

Table 1.1: Comparison of feature descriptors on the four datasets.

ORB and SIFT produced the most useful results. ORB generated the highest number of good matches after increasing the maximum number of detected features, while SIFT provided stable results with fewer features. AKAZE produced fewer correspondences in all scenes. The quality of the matches should be evaluated not only by the number

of matches, but also by the ratio of good matches and by geometric verification using RANSAC inliers.

1.4 Camera Calibration

Camera calibration was performed using the provided chessboard images, as the scene images are not suitable for intrinsic calibration. Chessboard corners were detected and refined before estimating the intrinsic camera parameters and distortion coefficients with OpenCV's calibration functions. To determine the influence of the number of calibration images, the calibration was repeated with an increasing number of images.

Program 1.4: Camera calibration using chessboard images

```
found, corners = cv2.findChessboardCorners(gray, chessboard_size)

if found:
    corners = cv2.cornerSubPix(
        gray, corners, (11,11), (-1,-1), criteria
    )

    object_points.append(objp)
    image_points.append(corners)

error, K, dist, _, _ = cv2.calibrateCamera(
    object_points,
    image_points,
    gray.shape[::-1],
    None,
    None,
)
```

Program 1.5: Evaluation of the required number of calibration images

```
for i in range(2, len(calibration_images) // 2 + 1):
    error, _, _, _, _ = calibrate(calibration_images[:i])
    print(f"Reprojection error with {i} images: {error:.4f} pixels")
```



```
Reprojection error with 2 images: 0.8000 pixels
Reprojection error with 3 images: 0.6925 pixels
Reprojection error with 4 images: 0.6132 pixels
Reprojection error with 5 images: 0.5578 pixels
Reprojection error with 6 images: 0.5189 pixels
Reprojection error with 7 images: 0.4905 pixels
Reprojection error with 8 images: 0.4682 pixels
Reprojection error with 9 images: 0.4527 pixels
Reprojection error with 10 images: 0.4527 pixels
Reprojection error with 11 images: 0.4336 pixels
Reprojection error with 12 images: 0.4183 pixels
Reprojection error with 13 images: 0.4223 pixels
Reprojection error with 14 images: 0.4087 pixels
```

Figure 1.3: Reprojection error for an increasing number of calibration images.

Calibration was successful using the provided chessboard images. As shown in Figure 1.3, at least three images were required to obtain a reliable calibration. Increasing the number of images reduced the reprojection error from 0.6925 pixels to 0.4087 pixels, while the improvement became smaller with additional images.

1.5 Extrinsic Estimation

The relative camera pose was estimated from two consecutive RGB images. SIFT was used to detect and match feature points. The essential matrix was computed using `findEssentialMat()`, and the relative rotation and translation direction were recovered with `recoverPose()`. The provided intrinsic camera calibration was used for pose estimation.

Program 1.6: Relative pose estimation

```
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

matches = cv2.BFMatcher().knnMatch(des1, des2, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

E, _ = cv2.findEssentialMat(pts1, pts2, K)

_, R, t, _ = cv2.recoverPose(E, pts1, pts2, K)
```

```
Rotation matrix:
[[ 0.90984978 -0.00178739  0.41493395]
 [-0.00111064  0.99997665  0.00674292]
 [-0.41493632 -0.00659588  0.90982655]]
Translation direction:
[[-0.98401138]
 [-0.04611656]
 [ 0.17203161]]
```

Figure 1.4: Estimated rotation matrix and translation direction.

The relative rotation and translation direction were successfully estimated. The quality of the estimation depends on the accuracy of the feature matches and can be evaluated using the number of RANSAC inliers.

1.6 Conclusion

The implemented computer vision methods were successfully applied to the drone datasets provided. RGB panorama stitching produced reliable results, while feature matching showed that SIFT and ORB provided the most suitable descriptors for thermal-to-RGB matching. Camera calibration was successfully performed using chessboard images, achieving a low reprojection error. Finally, the relative camera pose was estimated using feature correspondences and the essential matrix.

Chapter 2

Exercise 2

2.1 Introduction

This exercise investigates different approaches for image compression. The main focus is on compression of the frequency domain using the Discrete Cosine Transform (DCT), where the effect of removing frequency coefficients on image quality and compression ratio is analyzed.

Additionally, Implicit Neural Representations (INRs) are discussed as an alternative approach for image representation and compression.

2.2 Frequency Domain Compression

2.2.1 Implementation

The DCT compression algorithm was implemented using OpenCV. The main steps are shown below:

Program 2.1: DCT image compression

```
def dct_compress(image, keep_ratio=0.25):
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY).astype(np.float32)
    dct = cv2.dct(gray)

    h, w = dct.shape
    keep_h = int(h * np.sqrt(keep_ratio))
    keep_w = int(w * np.sqrt(keep_ratio))

    compressed = np.zeros_like(dct)
    compressed[:keep_h, :keep_w] = dct[:keep_h, :keep_w]

    reconstructed = cv2.idct(compressed)
    reconstructed = np.clip(reconstructed, 0, 255).astype(np.uint8)

    return reconstructed, compressed
```

The function first converts the RGB image to grayscale and applies the DCT. Compression is achieved by keeping only the low-frequency coefficients in the upper-left part of the DCT matrix. The remaining coefficients are discarded by setting them to zero. Finally, the inverse DCT reconstructs the compressed image. The effect of this compression process is illustrated in Figure 2.1.

2.2.2 Results and Analysis

The quality of reconstruction was first evaluated using a fixed value `keep_ratio` of 25%. Figure 2.1 shows the original grayscale image, the reconstructed image after DCT compression, and the retained DCT coefficients.

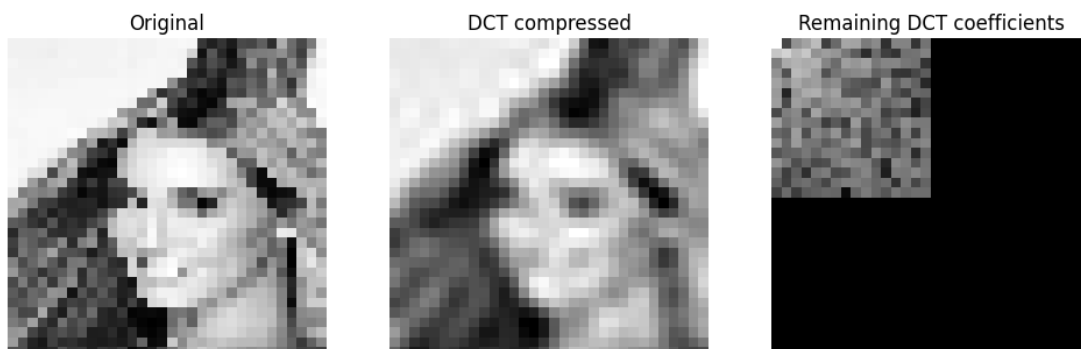


Figure 2.1: Comparison between the original image, the DCT compressed reconstruction using `keep_ratio = 0.25`, and the retained DCT coefficients.

The reconstructed image preserves the main structure of the face, while some high-frequency details are removed due to the discarded coefficients. This shows that the low-frequency components contain important information for reconstructing the overall appearance of the image.

To further analyze the influence of compression strength, different values of `keep_ratio` were tested (5%, 10%, 25%, 50% and 75%). The reconstructed images and their corresponding MSE values are shown in Figure 2.2.



Figure 2.2: Reconstructed images for different `keep_ratio` values. Lower values result in stronger compression and higher reconstruction error.

Removing more DCT coefficients increases compression, but reduces reconstruction quality. Lower `keep_ratio` values lead to a more visible loss of detail, blurrier images, and higher MSE values. Higher values preserve more information and result in better visual quality, but provide less compression. A medium `keep_ratio` provides a good balance between compression and reconstruction quality.

The theoretical compression ratio depends on the number of retained DCT coefficients. For example, keeping only 5% of the coefficients results in approximately a 20:1 compression ratio, while keeping 50% results in 2:1 compression. Therefore, `keep_ratio` provides direct control over the trade-off between storage reduction and image quality.

2.3 Implicit Neural Representations

2.3.1 Concept

Implicit Neural Representations (INRs) represent images using neural networks instead of directly storing pixel values. The network learns a function that maps the coordinates of the image (x, y) to the RGB values. The network parameters are then used as the image representation.

The size of the network influences the trade-off between compression and reconstruction quality. Larger networks can represent more details and improve reconstruction quality, but require more parameters and therefore more storage. Smaller networks provide stronger compression, but may lose image details.

The number of training epochs also affects the quality of reconstruction. More epochs usually improve the result, but increase the required computation time. Positional encoding helps INRs capture high-frequency details such as edges and textures, resulting in sharper reconstructions.

2.3.2 Comparison

Compared to frequency-domain methods, INRs use a neural network representation instead of storing frequency coefficients. DCT-based compression is usually faster because reconstruction only requires mathematical transformations, while INRs require the evaluation of the neural network.

INRs can represent fine details more flexibly, especially when positional encoding is used. However, this flexibility comes with higher computational costs. Frequency-domain methods are simpler and faster, but removing high-frequency coefficients can lead to loss of image details.

2.4 Conclusion

This exercise investigated image compression using frequency-domain methods and implicit neural representations.

DCT-based compression showed that reducing the number of frequency coefficients can significantly decrease the required storage while preserving the main structure of the image. The parameter `keep_ratio` provides direct control over the trade-off between compression ratio and reconstruction quality. However, removing too many coefficients leads to increased MSE and loss of image details.

INRs provide a different approach by representing images through neural network parameters instead of frequency coefficients. They can capture complex details, especially when using positional encoding, but require higher computational effort for training and reconstruction.

Overall, both approaches involve a trade-off between compression efficiency and image quality. DCT offers a simple and fast compression method, while INRs provide a more flexible representation with increased computational requirements.

Part II

Related Work Research - Real time Hand Gesture Recognition

Chapter 3

Introduction

In recent years a number of advancements in processing and synthesising human language in spoken and written form have been made. Applications like automatic subtitles and AI voice overs have found wide adoption on social media platforms with users generally accepting the technology and accessibility it provides. The focus so far has been on the conversion between aural and written representations of language. Naturally the question arises if such translations are also possible between other modalities of communication.

Sign language is the primary form of communication for deaf and hard of hearing people, many of them considering it their native language. Communication occurs entirely through visual stimuli in the form of hand gestures, facial expressions and body language. Despite historic evidence of sign language dating back to the Paleolithic age, it has only been recognized in the field of linguistics in the 1960s [7], [4]. Prior to this the belief was, that signing prevented the development of spoken language and was therefore a hindrance to the education of deaf children, a stigma that has persisted to this day. These misconceptions combined with the low number of sign language speakers has led to the exclusion of people who rely on this form of communication from many aspects of society. Machine translation between aural speech and signs could aid in bridging this gap. This report will look into the current state of research in hand gesture recognition, which is a core part of many sign translation approaches, and provides an overview of state-of-the-art methods in the field of automatic sign language translation.

The following chapter reviews hand gesture recognition as the technical foundation that underpins these sign language translation methods.

Chapter 4

Hand Tracking

4.1 Overview

Gesture recognition is a central part of human-computer interaction. It allows hand movements to be used directly as input signals. Computer-vision-based approaches are particularly popular, since they need no special sensors and work with cameras that are already widely available. This makes them well suited for real applications such as virtual and augmented reality, sign language recognition, and general user interfaces [3, 6]. A remaining problem is that recognition often becomes unreliable under changing lighting, cluttered backgrounds, or partial occlusion [6].

Gesture recognition systems are usually split into two groups: vision-based and sensor-based approaches. Sensor-based methods, such as data gloves, electromyography, or inertial measurement units, give signals that barely depend on the environment, but they need extra hardware and are less convenient to use. Vision-based methods instead work contactless and feel more natural, but they are more sensitive to lighting, background, and occlusion [6, 9]. Since both approaches have different strengths and weaknesses, they are increasingly combined instead of being seen as competitors [10].

4.2 Vision-Based Recognition

4.2.1 Processing Pipeline

Vision-based systems usually follow the same basic pipeline: data acquisition, preprocessing to reduce noise and lighting variation, segmentation of the hand region, feature extraction, and finally classification [6]. Segmentation is especially important, since mistakes here carry over into every following step and lower the overall performance.

For feature extraction, both classic descriptors (Histogram of Oriented Gradients, Local Binary Patterns, geometric contour features) and learned deep features are used. Learned features tend to adapt better across different datasets. For classification, Support Vector Machines and Hidden Markov Models used to be the standard but have mostly been replaced by neural networks [6].

RGB-D cameras like Microsoft Kinect or Intel RealSense add depth information to this pipeline, which helps with segmentation and makes the system more robust against

lighting changes and cluttered backgrounds. Real-time performance, sensor accuracy, and generalization to new environments are still open problems [6].

4.2.2 From Hand-Crafted Features to Deep Learning

Early systems relied on manually defined features and did not generalize well outside controlled settings. Deep learning changed this by extracting features automatically from raw data. Convolutional Neural Networks are good at capturing spatial structure, while Long Short-Term Memory networks were initially used to model how gestures change over time [3]. More recently, transformer and attention-based models have started to replace recurrent networks, since they can model long-range dependencies and put more weight on relevant image regions, which improves performance on more complex gestures [9].

4.3 Sensor-Based Recognition

Surface electromyography (sEMG) is one of the main sensor-based alternatives to vision. It measures electrical muscle activity to figure out which gesture was performed. Since it does not depend on visual conditions, it is useful for applications like prosthetic control or wearable interfaces. Deep learning models can now work directly on raw sEMG signals, so manual feature engineering is no longer necessary. The main problem that remains is that muscle signals vary a lot between individuals, which makes it hard to build models that generalize across users [11].

4.4 Multimodal Fusion

To work around the limits of a single modality, more research combines RGB, depth, skeletal, and sEMG data into multimodal systems. The main benefit is robustness: if one sensor fails, for example vision under occlusion, another modality such as an inertial or muscle-based sensor can compensate [10]. The main difficulty is that these data sources are very different from each other in resolution, noise, and information content, which makes fusion non-trivial [9].

To fuse this data, hybrid deep-learning architectures are commonly used, combining CNNs for spatial features with recurrent or graph networks for temporal and structural dependencies. Transformers are increasingly used as well, since they can flexibly integrate different data sources [10]. Mahmud et al. show this in practice by combining depth and skeletal data for dynamic gesture recognition, reporting clear improvements over single-modality baselines, especially for gestures that look visually similar [5].

4.5 Open Challenges

Across vision-based, sensor-based, and multimodal systems, deep learning is now the common foundation, and the general trend is moving from single-modality pipelines toward multimodal fusion for better robustness. Challenges that keep showing up across

the literature are the lack of standardized, realistic benchmarks, limited real-time capability, and poor generalization to new users and environments [9, 10].

Chapter 5

Vision Based Sign Language Recognition

This chapter provides an overview of current applications of gesture recognition in sign language translation. The following section will elaborate on the task definition of sign language translation as well as highlight some challenges and how they are addressed with different technologies.

5.1 Overview

In the field of sign language translation there are two main goals. Sign language production [8] aims to generate visual output of a signing individual from an input phrase. Sign interpretation instead refers to the conversion of a video feed into either written or spoken output. This chapter focuses on interpretation. It can be further divided into *Continuous Sign Language Recognition* (CSLR) and *Sign Language Translation* (SLT), where the former aims to identify individual signs in a video feed and the latter aims to translate a full sentence into written or spoken language [2].

5.2 The Challenge of Sign Language Recognition

Each sign language features unique grammar, vocabulary, and syntax. They are complete languages and exist separately from the spoken languages they share a region with. Instead of a 1-to-1 mapping of signs to words, the meaning of a phrase needs to be understood in context in order to create a correct translation.

Signs are also composed of more than just hand gestures. The meaning of a phrase can change based on facial expression and the position of elements relative to the body and each other. These aspects are referred to as *non-manual markers* and ideally need to be captured alongside hand gestures for a system to translate correctly [4].

5.2.1 Datasets

Training and evaluating a model requires a dataset with annotations that can be compared against the model's translation. In addition to sentence-level translations, some older methods also require gloss-level annotation, which is considerably more time consuming to produce. Glosses are the written representation of a single sign within a

sentence and act as the building blocks from which a full sentence is later constructed.

The *RWTH-PHOENIX-Weather 2014T*¹ dataset was the first publicly available dataset for use in SLT and features German Sign Language (DGS) with gloss-level annotation. The *HOW2SIGN*² dataset instead contains American Sign Language (ASL) samples with translations in both spoken and written English. It also provides multiple camera angles of the same signer along with depth and skeletal data. This kind of multimodal data enables more robust recognition, similar to the multimodal approaches discussed in the previous chapter, and is increasingly what newer translation methods rely on.

5.3 Current Approaches

Generally, the problem of sign language translation can be split into three subtasks. First, the boundaries of individual sentences and statements need to be identified in the video feed. The lack of punctuation or shifts in tone of voice to mark such boundaries makes *Sign Segmentation* a non-trivial task. *Sign Recognition*, corresponding to CSLR, then describes the identification of individual linguistic elements from the video feed, where spatial context and non-manual markers need to be taken into account. Finally, the desired output format, usually text or speech, needs to be generated in the *Sign Translation* step [2].

Older approaches relied heavily on gloss representation as an intermediate step in this pipeline: a CSLR model first recognized the gloss sequence, and a separate text-to-text model then translated that sequence into spoken language. The following two subsections trace how this pipeline has evolved, first by learning recognition and translation jointly, and more recently by moving toward richer multimodal input processed through large pretrained language models.

5.3.1 Sign Language Transformers

The gloss-based two-step pipeline works reasonably well, but the intermediate gloss representation becomes a bottleneck, since the final translation can only be as good as the gloss recognition step, and glosses themselves are a simplified, incomplete representation of what is actually signed.

Camgoz et al. [2] address this with an architecture called Sign Language Transformers, which learns recognition and translation jointly instead of as two separate stages. The model consists of two connected transformer components: a Sign Language Recognition Transformer (SLRT), an encoder trained with a Connectionist Temporal Classification (CTC) loss to predict gloss sequences from video, and a Sign Language Translation Transformer (SLTT), a decoder that generates the spoken language sentence directly from the representations learned by the SLRT. Because both parts are trained together, the network still benefits from gloss-level supervision during training, but the final translation is no longer strictly limited to what the gloss recognition step can capture.

¹<https://github.com/enhuiz/phoenix-datasets>

²<https://how2sign.github.io/>

Evaluated on the PHOENIX14T dataset, this joint approach roughly doubled the translation performance of prior end-to-end video-to-text models and even slightly outperformed the earlier gloss-based translation approach that had been treated as an informal upper bound. This result supports the idea that direct video-to-text translation can eventually be more accurate than translation through a gloss intermediate, given a sufficiently capable model architecture and training setup.

5.3.2 Large Sign Language Models

Where Sign Language Transformers still operate on 2D video, more recent work has moved toward richer multimodal input and toward reusing pretrained large language models (LLMs) instead of training a translation model from scratch. Zhang et al. [12] propose Large Sign Language Models (LSLM), which translate American Sign Language using 3D motion data instead of 2D video. The 3D representation, based on the SMPL-X body model, captures body, hand, and face motion jointly and is more robust to camera angle, lighting, and background than flat video, addressing the same non-manual marker and spatial context requirements discussed earlier in this chapter.

This motion is compressed into discrete tokens using a vector-quantized autoencoder and then mapped into the same embedding space used by a pretrained LLM, allowing sign motion and text to be processed together. The model is first pretrained for basic sign recognition and then instruction-tuned, so it can follow different phrasings of the translation task rather than a single fixed prompt. LSLM outperforms a baseline built on a general-purpose motion-language model, showing that combining multimodal 3D input with the linguistic knowledge already present in pretrained LLMs is a promising direction, rather than training narrow translation models from scratch.

Taken together, these two approaches illustrate a broader shift in the field: away from brittle, gloss-bottlenecked pipelines and 2D-only input, and toward joint end-to-end training on richer, multimodal representations of sign.

Chapter 6

Conclusion and Outlook

This report has provided an overview of the current state of research in the field of hand gesture recognition and highlighted some of its applications in sign language translation. Examining the relevant literature revealed a trend towards deep learning-based approaches, which have yielded significant improvements in accuracy and robustness. Another common theme in both chapters was the use of multimodal data to make up for the limitations of a single video feed. Supplementary depth maps and skeletal data can make up for ambiguity caused by lighting and occlusion in the image.

Foregoing the gloss representation stage in sign translation has shown both promising results in accuracy as well as reduced the work required in creating suitable training data. For deep learning-based approaches, the availability of large, annotated datasets is crucial. The covered literature focuses on American and German sign language, which have a large number of speakers and thus a larger pool of data. Enabling translation of less popular sign languages, specific regional dialects and individual signing styles, requires a lower barrier for data collection and annotation.

Automated sign recognition and production has the potential to improve accessibility online and in real life. In education, new tools could be developed to complement traditional teaching methods, which would allow for wider adoption of the language. It is, however, important to note that sign language is traditionally taught by non-hearing individuals in an effort to preserve the culture and history of a language that has been historically marginalized [1]. Besides the technical challenges, the social and cultural implications of who gets to build and control these systems will become increasingly important as the technology matures.

References

- [1] *ASL Teaching Ethics*. <https://www.handspeak.com/learn/202/>. (Visited on 07/09/2026) (cit. on p. 23).
- [2] Necati Cihan Camgöz et al. “Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation”. In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2020, pp. 10020–10030. DOI: 10.1109/CVPR42600.2020.01004 (cit. on pp. 20, 21).
- [3] Cui Cui, Mohd Shahrizal Sunar, and Goh Eg Su. “Deep Vision-Based Real-Time Hand Gesture Recognition: A Review”. *PeerJ Computer Science* 11 (June 2025), e2921. DOI: 10.7717/peerj-cs.2921 (cit. on pp. 17, 18).
- [4] Susan Goldin-Meadow and Diane Brentari. “Gesture, Sign, and Language: The Coming of Age of Sign Language and Gesture Studies”. *Behavioral and Brain Sciences* 40 (2017), e46. DOI: 10.1017/S0140525X15001247 (cit. on pp. 16, 20).
- [5] Hasan Mahmud, Mashrur M. Morshed, and Md Kamrul Hasan. “A Deep Learning-based Multimodal Depth-Aware Dynamic Hand Gesture Recognition System”. *The Visual Computer* 40.1 (Jan. 2024), pp. 11–25. DOI: 10.1007/s00371-022-02762-1. arXiv: 2107.02543 [cs.CV] (cit. on p. 18).
- [6] Jing Qi et al. “Computer Vision-Based Hand Gesture Recognition for Human-Robot Interaction: A Review”. *Complex & Intelligent Systems* 10.1 (Feb. 2024), pp. 1581–1606. DOI: 10.1007/s40747-023-01173-6 (cit. on pp. 17, 18).
- [7] Robert J. Ruben. “Sign Language: Its History and Contribution to the Understanding of the Biological Nature of Language”. *Acta Oto-Laryngologica* 125.5 (May 2005), pp. 464–467. DOI: 10.1080/00016480510026287 (cit. on p. 16).
- [8] Ben Saunders, Necati Cihan Camgoz, and Richard Bowden. “Signing at Scale: Learning to Co-Articulate Signs for Large-Scale Photo-Realistic Sign Language Production”. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, June 2022, pp. 5131–5141. DOI: 10.1109/CVPR52688.2022.00508 (cit. on p. 20).
- [9] Jungpil Shin et al. “A Methodological and Structural Review of Hand Gesture Recognition Across Diverse Data Modalities”. *IEEE Access* 12 (2024), pp. 142606–142639. DOI: 10.1109/ACCESS.2024.3456436 (cit. on pp. 17–19).

- [10] Shushen Wang et al. “A Comprehensive Survey of Multi-Modal Manipulative Hand Gesture Recognition: Sensors, Datasets, and Models”. *Journal of Visual Communication and Image Representation* 119 (Aug. 2026), p. 104840. DOI: 10.1016/j.jvcir.2026.104840 (cit. on pp. 17–19).
- [11] Marcello Zanghieri. *sEMG-based Hand Gesture Recognition with Deep Learning*. June 2023. DOI: 10.48550/arXiv.2306.10954. arXiv: 2306.10954 [eess.SP] (cit. on p. 18).
- [12] Sen Zhang et al. *Large Sign Language Models: Toward 3D American Sign Language Translation*. Nov. 2025. DOI: 10.48550/arXiv.2511.08535. arXiv: 2511.08535 [cs.CV] (cit. on p. 22).